BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
26 October 2026

Abstract

Agents exploring alternatives from a common revision need private working state and continued access to shared inputs. Sharing file contents avoids duplication, but leaves the costs of constructing private views, serving metadata, and fetching partial data. BrewFS addresses these costs through three independently optional mechanisms. Delta-Based Workspaces keep a shared immutable baseline and store private namespace state and data-range mappings, preserving references to unchanged data. Packed Metadata moves supported queries for a selected, published read-only directory version into authenticated object pages that co-locate names, attributes, and data locators. It targets stable small-file collections reused across tasks, including training inputs. Variable-Granularity Object Storage separates client object formation from backend encoding and verification units to control the work induced by partial reads. These choices incur distinct preparation, access, and retention costs. We describe their representations and update protocols, and characterize 1,138,884 FUSE requests from eight coding-agent runs. The evaluation plan tests the expected benefits in each mechanism's intended workload, then examines fork, migration, and concurrent execution.

Draft status. The workload characterization is complete. The formal system-performance study and end-to-end recovery validation in Section 8 remain to be conducted.

1 Introduction

Coding agents traverse repositories, inspect files, edit code, and run builds and tests [24, 27]. A service that launches several attempts from the same revision must give each one a consistent starting view and private changes, and must keep saved results available to later attempts or to another worker. Storage therefore has to serve names, attributes, and file data for a workspace while that workspace changes underneath it.

Consider attempts that reuse a large source tree or dataset while changing a small part of their working state. Copying the tree repeats preparation work; sharing its contents still leaves names, attributes, and data mappings to serve. In a database-backed file system, queries over fixed inputs can compete with mutable updates. Partial reads incur a further cost when the client or storage backend retrieves and verifies more bytes than requested. A cheap fork therefore does not by itself make the resulting workspace cheap to use.

Machine-learning agents provide a useful example [3]. Several experiments may reuse a fixed training dataset while changing their code and writing separate checkpoints and outputs. With many small input files, the shared namespace remains substantial even though its contents are read-only. Its representation can be chosen separately from that of the agents' mutable working state.

Prior systems already provide much of the foundation. OverlayFS and BranchFS isolate private changes over shared state [13, 22]; DeltaFS publishes private namespace updates as immutable logs [28]. IndexFS and InfiniFS improve metadata organization, distribution, and caching [14, 19]. CernVM-FS and Nydus serve immutable file trees with indexed metadata and on-demand data access [2,15], and CoFS exploits a fixed image namespace to index metadata for kernel lookup [23]. An object-backed workspace built on these approaches still has to choose how to represent private updates, which fixed metadata to serve outside the mutable database, and which units to fetch and verify.

Our workload study shows that the access side deserves that attention. Across 1,138,884 FUSE requests from eight coding-agent runs, metadata methods accounted for 57.3% of all calls. Request lengths varied over two orders of magnitude and calls arrived in bursts (Section 2.3). The runs characterize coding workloads; they do not measure machine-learning data access.

BrewFS therefore separates the identity of shared state from its metadata representation and from the units in which payload bytes are fetched. Three mechanisms implement this separation:

- **Delta-Based Workspaces** retain private namespace

changes and data-range mappings over a pinned immutable baseline. For attempts that change a small part of a shared tree, unchanged ranges retain their references, avoiding whole-file copy-up on those updates. The lower-query interface accepts native or packed metadata (Section 4).

- **Packed Metadata** serves an explicitly built and published read-only directory version through authenticated object pages whose directory-name groups colocate entries, common attributes, and data locators. Repeated reads of a fixed small-file collection can thus serve supported lower queries without per-file database access. Private-upper checks, mutable updates, and lifetime control retain their existing services (Section 5).
- **Variable-Granularity Object Storage** separates client object formation from backend encoding and verification units. For small reads within larger objects, these choices control how much extra data is fetched and verified. A finalized object can carry an encoding-layout hint; the backend validates and persists the effective layout for subsequent reads and repair (Section 6).

Each mechanism is independently optional. A workspace can keep its baseline in native metadata; a packed dataset can serve standalone read-only access without any workspace; an ordinary volume can configure object granularity. A native overlay forks without paying the construction cost of a packed version, and a published pack is reused by later children instead of being rebuilt.

This paper presents the design, the implementation scope, and a completed workload characterization. The evaluation plan (Section 8) first asks where each mechanism should pay off, then examines fork, migration, and concurrent execution.

2 Background and Motivation

2.1 Private work and shared inputs

An agent's workspace contains source files, dependencies, generated artifacts, and temporary files. Two attempts starting from the same revision may read the same file and then make incompatible edits. Each must continue to see its own version, including after deletion or truncation. A stable baseline identity and private changes provide this isolation independently of where metadata and bytes reside.

Read-only inputs have a different lifetime from working state. In the machine-learning scenario, an experiment reads a fixed dataset while writing model checkpoints and transformed outputs elsewhere. A directory-based image loader, for example, enumerates a dataset to build a sample list, then opens selected files during loading [21]. Separate experiments repeat this discovery work against the same version, so the namespace of that dataset is read repeatedly while the data itself never changes.

Two properties make such a collection different from ordinary working state. Its content is fixed for the lifetime of a version, so its records can be organized once and reused by every reader. And its metadata occupies a larger fraction of its representation: every file requires a name, attributes, and a data locator regardless of its payload length.

2.2 From file operations to storage work

Reading a file requires path resolution, a data mapping, and access to the referenced bytes. Metadata services batch and cache records [18, 19], while immutable distributions organize them for demand fetching [2, 15]. A file-system call therefore need not cause a remote metadata request. The relevant question is how much database traffic remains under the actual cache and concurrency conditions.

Payload access introduces another translation. Kernel caching and readahead determine which requests reach FUSE; the client then fetches object ranges or complete objects. Encoding and integrity checks may require still larger physical reads [1, 6]. Writes pass through buffering and object formation before publication. FUSE request lengths thus describe neither application syscall sizes nor physical storage units, and a mechanism that changes one layer does not by itself change the others.

2.3 Observed requests in coding-agent runs

Capture. We reanalyzed eight SWE-bench Verified task runs [7]. All used the same harness configured for `deepseek-v4-flash`. Three runs had a broader investigation and testing prompt and a 30-minute budget; all finished earlier. Each base-commit checkout used an instrumented passthrough file system on Ubuntu 24.04 under WSL. Instrumentation recorded requests reaching FUSE handlers.

The capture contains 1,138,884 requests across all eight runs. Counts include all statuses, 64 setup/canary requests before startup, and teardown, including 24 requests after recorded run completion. Pooled statistics weight requests equally rather than tasks. The companion workload record retains instrumentation versions, per-task counts, and source-validation details.

Metadata traffic. Six metadata methods (`lookup`, `getattr`, `getxattr`, `readdirplus`, `opendir`, and `releasedir`) accounted for 57.3% of all calls (Figure 1(a)). Other contains methods outside these six and read/write. Among successful events, the metadata share was 45.8%, showing the contribution of unsuccessful probes. Across individual tasks, all-status shares ranged from 39.6% in SymPy to 65.4% in Django. Read calls outnumbered

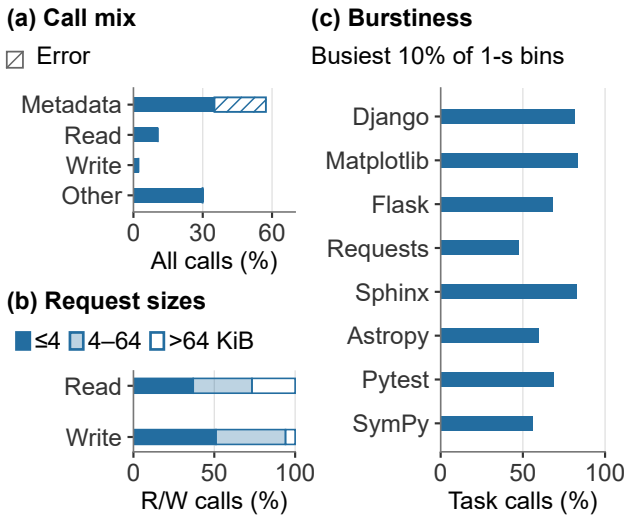


Figure 1: FUSE requests across eight coding-agent runs. (a) Pooled call shares; hatching marks errors within each category. (b) Pooled requested-length distributions, normalized separately for reads and writes. (c) Each task's call share in its busiest 10% of one-second windows. Panels (a,b) weight requests equally.

writes by 4.87:1 overall; per-task ratios ranged from 3.12:1 to 42.7:1.

Request sizes. All 120,434 reads and 24,712 writes succeeded. Reads ranged from 4 to 128 KiB, with a median of 12 KiB. Writes ranged from one byte to 128 KiB, with a median of 4 KiB. Requests of at most 4 KiB accounted for 37.0% of reads and 51.0% of writes. The interval $(4, 64]$ KiB contained a further 36.3% of reads and 43.0% of writes (Figure 1(b)). This mixture calls for measuring how fetching and object formation transform requests before selecting storage granularity.

Variation across tasks and time. Tasks generated 8,542–634,726 calls over capture spans of 42.7–556.1 seconds. Astropy contributed 55.7% of all requests, so the pooled distributions describe recorded traffic rather than an equally weighted typical task. The busiest 10% of one-second windows contained 47.4–83.7% of each task's calls (Figure 1(c)). Windows include idle and final partial windows; the number selected is rounded up per task. These differences favor retaining per-task mixes and timing in replay. Each task was observed once, and repositories, prompts, and budgets differed; the observations do not isolate their effects.

Measurement limits. The capture contains 872,519 `ok`, 253,669 `error`, and 12,696 `incomplete` events. Errors came from `lookup` (233,452), `getxattr` (20,203), and `rmdir` (14). Missing errno values prevent attribution of individual errors. Incomplete events lack a recognized paired completion: 12,688 are `forget` records and eight are `destroy` records. All agent and daemon exit codes were zero; task-grading success was not established.

Handler intervals measure WSL passthrough activity, not remote storage latency or model response time. Capture spans include inter-request gaps; overlapping handler intervals cannot be summed into task duration. The traces establish request mix and timing for this panel. They do not measure ML workloads, cross-task sharing, sparse mutation, or BrewFS performance.

2.4 Design implications

The coding traces supply the access shape that the mechanisms must fit: many metadata calls, mixed request lengths, and bursts of activity within a task. Baseline reuse is a separate workload assumption: attempts that start from the same tree and modify a small part can retain references to unchanged state. The traces do not measure that sharing or modification pattern.

The machine-learning scenario adds a complementary case: a fixed collection of many small files, read repeatedly by many consumers, whose namespace should not compete with the mutable updates of the writers that use it. Together they point to three separable cost reductions, and Section 3 describes how BrewFS keeps workspace state, metadata representation, and payload layout apart so that each can be chosen on its own terms.

3 System Overview

3.1 State, metadata, and payloads

BrewFS separates three decisions along the file-access path: which version a task sees, how it retrieves immutable metadata, and how it stores and fetches payloads. A lower reader resolves names, attributes, and data locators under a fixed version identity. The data path returns the referenced logical bytes using each object's persisted layout. These interfaces allow metadata representation and object layout to change without changing the file view.

Figure 2 maps the design to the following sections: shared state and private changes (Section 4), native or packed metadata (Section 5), and object formation separated from backend layout (Section 6). The mechanisms are independently optional, so an ML experiment can mount a standalone packed dataset beside a native writable workspace.

3.2 Pinning a baseline

A workspace binds a shared immutable lower and a private writable upper. The lower supplies a fixed version through a sealed native layer or a packed manifest. It may cover

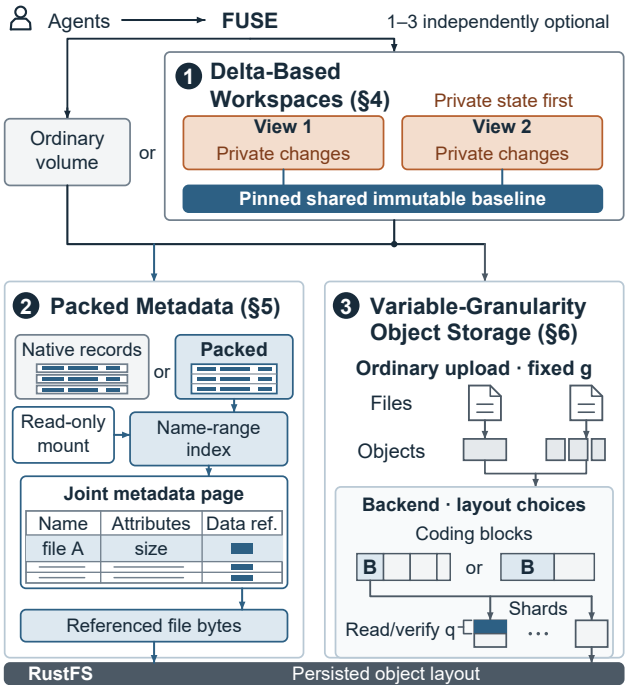


Figure 2: The three optional mechanisms and their corresponding sections. Workspaces retain private changes over shared state; Packed Metadata serves a selected read-only subtree. Object granularity separates client block size g, backend encoding block size B, and shard verification quantum q. Layouts are schematic. Implementation coverage is described in Section 7.

a selected subtree; the rest of the source volume need not be frozen. Resolving every lookup through a mutable latest-version pointer would violate this fixed starting view.

Private records determine which inherited names and ranges remain visible. Section 4 gives the lookup and update rules; merging independent edits requires an application protocol.

3.3 One access, three boundaries

Consider an agent that opens `config.json` in a shared dataset and reads its first 4 KiB. Three decisions determine what that read costs. The name is resolved against the workspace's private upper and, if no private record exists there, against the lower; the lower answers through a native catalog or through a joint page fetched from object storage. The resulting record supplies a data locator, and the client turns it into an object reference and a byte range using the geometry in which that object was formed. The backend finally reads and verifies the shard interval to which the range maps.

3.4 Reads, writes, and publication

An overwrite stores new bytes and publishes a private mapping, leaving other workspaces' references intact. Buffered writes can satisfy local reads before their objects become remotely durable.

Publishing a recoverable version requires durable referenced objects and validated metadata. Ownership records retain metadata and payloads while workspaces or readers still need them. Section 7 describes publication and runtime recovery.

4 Delta-Based Workspaces

4.1 A shared baseline and private changes

Delta-based workspaces retain a shared immutable baseline and private records for modified names, attributes, deletion markers, and data mappings. This avoids materializing the baseline for every attempt. Private records hold current state, including complete attributes where needed; data-range mappings preserve references to unchanged bytes.

Workspace creation registers the immutable base and the private control state without copying the metadata tree. The catalog identifies the base by layer, sealed version, and root hash; a packed lower also binds its authenticated manifest. Creation still pays for binding validation, ownership records, and mount preparation. The lower can use native metadata or an already published pack (Section 5).

4.2 Resolving names

Lookup first checks the private upper. An entry supplies the result; a deletion marker reports absence and stops the lookup. Otherwise, the reader consults the lower unless an opaque directory or another inheritance rule hides the name. Those rules suppress inherited entries, leaving explicit upper entries visible. Deleting an inherited name must retain its marker, since removing the upper record alone would expose the lower name again.

Directory enumeration applies the same precedence and removes duplicate names. Both the buffered and direct-I/O workspace paths preserve an inherited file's inode number when materializing private attributes or mappings. Updates through one hard-link alias therefore remain visible through the others.

The direct-I/O path keeps private namespace records in transactions. Rename locks participating inodes in number order, validates the observed records and writer authority, and commits the source deletion marker and destination binding atomically. The transaction also updates affected inode and parent records. Namespace reads validate an operation-local view and retry conflicts, so they cannot return partially applied rename records. Separate calls may observe opposite

sides of the commit. The buffered path instead serializes local mutations and uses version-guarded catalog transactions. Neither provides a snapshot across multiple system calls.

4.3 Overwrites and truncation

An overwrite changes the data references for a file range. The buffered path records private extents alongside namespace and inode changes. The direct-I/O path represents each changed logical chunk with a normalized map of upper data, lower references, and zero-filled holes. Within the range a map covers, every interval names exactly one source, so no part of it refers to the lower by accident. A range with no private override at all continues to inherit the lower's bytes, and the inherited boundary only shrinks, so an attempt never exposes more of the baseline than it started with.

A short overwrite replaces its interval and retains lower references elsewhere. Truncation must invalidate both inherited and previously written bytes. The direct-I/O path first commits the reduced size, a nonincreasing lower-visible boundary, and a cleanup intent. Cleanup replaces discarded spans with zeros; extension completes pending cleanup before increasing the size. For example, writing $[6, 10)$, shrinking to 7, and extending to 34 must leave zeros throughout $[7, 34)$. Restricting only the visible lower size would expose the old upper bytes.

The buffered client's dirty overlay serves local writes before writeback. If dirty data covers $[4, 8)$ KiB, a read of $[2, 10)$ KiB combines two committed 2 KiB ranges with the dirty 4 KiB range. Persistent private mappings and this local overlay can coexist. Their durability differs, and a short logical write need not produce a short stored object.

4.4 Publication and retention

The direct-I/O path uploads an object durably before publishing its mapping. Ordinary buffered writeback also defaults to upload-before-commit, but offers an explicit commit-before-upload mode. That mode can expose mappings before their objects are remotely available. Preparing a recoverable version must therefore drain and validate referenced objects, even when local reads already return the new data.

Worker handoff requires stopping source writes and making payloads and metadata durable before publishing the version. The destination binds that version and acquires write authority; lease and generation checks must reject stale writers. Applications checkpoint execution state and reopen files separately, because a published file version carries file contents and names, not process memory or open handles.

Reclamation retains every object still referenced by a workspace or reader and distinguishes these objects from abandoned construction output. The metadata footprint consists of the shared baseline, each workspace's private records and mappings, and binding and ownership records. Private state grows with modifications and deletions. Physical accounting also includes payloads, encoding and checksum overhead, unreclaimed objects, and caches.

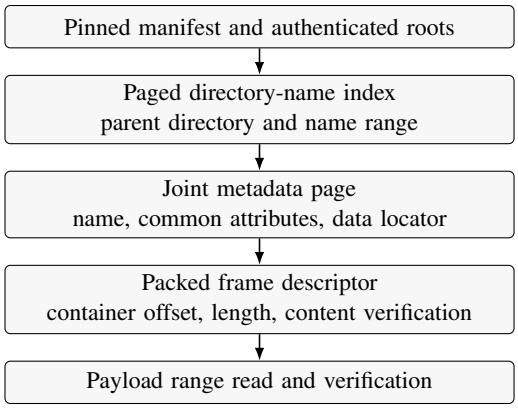


Figure 3: Packed-snapshot lookup and data access. A name lookup returns joint metadata and a data locator. A file read resolves authenticated descriptors before fetching and verifying the referenced payload range.

5 Packed Metadata

Repeated access to a fixed small-file collection can place read traffic on the metadata service handling mutable updates. Packed Metadata organizes that version into object pages, moving supported lower queries to object storage and client caches. This targets database load and interference; construction and publication are paid once per version and amortized over its readers.

5.1 What is packed

Packed Metadata applies to a selected read-only directory version that has been explicitly built and published. It can serve a standalone read-only mount or supply a workspace lower. Writes in the latter case go to the private upper; they never modify packed pages. Files outside the published version receive no benefit from this representation.

A logical read group contains entries from one parent directory over a range of raw names in byte order. Its joint metadata page co-locates each entry with common attributes and data locators. A name lookup can therefore supply information needed by subsequent attribute and data operations. Grouping neighboring names also makes the page a natural unit for reuse; Section 7 describes current caching.

Raw pages are bounded at 256 KiB. A physical container may hold several groups, so the metadata access unit need not equal the stored object. Range indexes route queries to groups; index height depends on page capacity and record count rather than source-directory depth.

5.2 Querying a pinned snapshot

A reader pins a manifest and starts from its authenticated index roots. For a name lookup, the directory-name index identifies the group covering the key. The reader resolves the container and fetches the metadata page's byte range. Before decoding, it verifies the stored-byte digest; after decoding, it checks entry counts and name boundaries against the authenticated group reference. The matching record supplies the inode, common attributes, and data locator (Figure 3).

Other queries use indexes suited to their keys. Attribute queries addressed by inode use a separate index. Enumeration traverses ordered name ranges, with counted indexes supporting directory-ordinal navigation. A file read returns inline contents or follows a locator to an authenticated descriptor and verified payload range. Packed data frames define snapshot-level access and integrity units; the backend serves them using its own persisted object layout.

Every returned record or payload must connect through authenticated references to the pinned manifest. Missing objects and failed authentication are errors, not evidence that a name is absent. Cached records remain valid for that identity after newer versions appear. A cold lookup may still require several index requests; data reads can add descriptor and payload requests. Co-location provides opportunities for reuse without fixing the number of GETs per operation.

Supported lower queries need no per-file record in the mutable database. Private-upper checks, mutations, and ownership coordination continue through the workspace and control services.

5.3 Building and publishing a version

Construction starts from a consistent source view, established by a snapshot or validation protocol. Immutable output alone cannot establish source consistency. The producer groups records, uploads objects, and builds the indexed manifest. Publication then installs the binding after the referenced objects are durable and valid. Retained versions and readers keep their objects live until reclamation is safe.

If no published pack exists, the first packed fork additionally waits for construction, upload, validation, and publication. A native overlay requires native version capture and binding but avoids pack construction. Later children reuse the published pack rather than rebuilding it. Query offload can help while the original database records are retained; reducing their persistent footprint also requires safe retirement, which is a separate step from the offload itself.

The relevant cost therefore spans construction, each use, retention, and reclamation. Grouping gives related queries a common page to reuse; the available caches determine whether that reuse occurs. Isolated lookups may transfer unused records. Inlining, compression, and payload co-packing introduce additional effects that must be measured separately. Long retention or rapid version churn can outweigh access savings even when queries become faster.

A fixed ML dataset illustrates the intended use. Several experiments can mount the same packed input while keeping code changes, checkpoints, and temporary results in writable storage, so the build cost is paid once per dataset version and amortized over its readers. Section 8 compares native and packed representations over this full lifecycle.

6 Variable-Granularity Object Storage

Small reads within large objects can fetch and verify unused bytes. Smaller objects reduce that opportunity but add requests and per-object overhead; smaller verification units add checksums. BrewFS separates client object formation from backend encoding and verification so these costs can be controlled at their respective layers. The finalized object connects them: its length is known after assembly, compression where applicable, and framing.

6.1 From file ranges to objects

The ordinary data path maps file offsets to logical chunks and resolves the visible slices within each chunk. A chunk is an addressing range; slices identify the content supplying its visible intervals. The uploader partitions a slice using client block size g and addresses each block by slice identity and block index. A 10 MiB slice with $g = 4\,\text{MiB}$, for example, can produce raw blocks of 4, 4, and 2 MiB.

Compression and framing determine the finalized upload length L_o. Thus g partitions the slice, while L_o measures the bytes uploaded. Flush boundaries and partial writes affect object formation, and a tail need not occupy a full block. Readers use the same geometry to translate slice intervals into object references and offsets. Replacing one interval can leave untouched intervals referencing older objects.

Chunk size must be a nonzero multiple of g. Because readers reconstruct addresses from this geometry, changing g for existing data requires a compatible representation or migration. It is a configuration choice, rather than a controller that adapts to observed access.

6.2 Backend encoding and verification

Once L_o is known, the client can pass an optional encoding-block-size hint to a compatible RustFS research backend. The hint requests backend erasure-coding block size B without changing g or multipart part sizes. The backend checks eligibility and compatibility, then persists the effective B. Omitting a hint retains default selection. Multipart initiation uses the complete object's length, and retries retain that object's decision.

The interface accepts fixed hints of 64 KiB, 256 KiB, 1 MiB, and 4 MiB, or a length-based policy. These choices

g	Object count	Whole-object bytes fetched
1 MiB	64	2 MiB
4 MiB	16	4 MiB
8 MiB	8	8 MiB
16 MiB	4	16 MiB

Table 1: Arithmetic example of object formation and a 2 MiB partial read. Complete intersecting objects are fetched, with no caching or compression and ignoring framing overhead. Range reads can transfer less.

apply to the formed object. Section 7 describes the current policy and its relationship to the backend default.

The physical verification/read quantum q determines how much shard data is verified together. A separate backend policy can select smaller units for eligible new framing. Full stripes must contain a whole number of verification units per shard, with coding blocks aligned across all data shards. Hint admission and new verification framing have separate compatibility gates. Reads and repair use persisted geometry even after write settings change.

6.3 Physical work for a partial read

Range GET can retrieve part of an object when the format permits independent access. The ordinary store supports page-aligned range reads for eligible small requests; compressed blocks or whole-object fetching can require larger transfers. Packed frames provide a further decoding and integrity boundary inside snapshot objects. Their size is distinct from both g and q. Caching and concurrent-read coalescing determine which requests reach the backend.

The backend maps an object range to shard intervals and rounds each interval outward to verification-unit boundaries. A 2 KiB shard read at offset 3 KiB, for example, touches two 4 KiB units: 8 KiB plus checksums, before tails or batching. Client fetching, compression, and coding determine the shard interval. Smaller q can reduce over-read but adds checksum overhead. With unchanged data and parity shard counts, nominal redundancy remains constant; a large B does not imply a fully padded tail stripe.

6.4 Granularity tradeoffs

Table 1 illustrates the tradeoff in g for an uncompressed 64 MiB slice starting at offset zero. Increasing g reduces object count but increases transfer for a read of $[10, 12)$ MiB when complete objects must be fetched.

These layers require distinct measurements. Network read amplification compares object-response bytes with application bytes served; physical read amplification uses storage-device bytes as the numerator. Upload amplification compares uploaded bytes with application bytes written. Request count, compression work, and cache occupancy capture additional costs. Compaction, retries, and short slices can increase upload work, so smaller objects need not reduce write amplification.

A smaller backend quantum helps only if client fetching exposes sufficiently small ranges. Snapshot producers and workspace upper writers also have their own upload formats. The implementation differences below determine which combinations can be compared without changing the client path.

7 Implementation

BrewFS provides buffered, direct-I/O, and standalone read-only paths. Their cache, writeback, and fetch policies determine which mechanism effects each can expose.

7.1 Access paths

Ordinary volumes use mutable metadata and buffered writeback. Buffered workspaces add private-state resolution over a sealed catalog or packed lower. Standalone packed mounts omit the private upper; their entry currently requires zero payload-cache and prefetch budgets.

The authenticated packed reader caches index pages, per-name results including absence, and attributes. Directory enumeration retains and reuses the decoded joint page for its current group. Independent name lookups fetch and decode that page on a per-name cache miss. General sharing of decoded pages, partial decoding, and incremental rebuilding remain optimization opportunities.

Direct-I/O workspaces store private namespace and mapping records transactionally, implementing the rename and truncation rules in Section 4. They bypass buffered writeback and metadata time-to-live caching, with admission bounds on operation sizes, enumeration, and ownership. This path supports fresh workspaces without process-crash takeover. It authenticates an entire upper object before returning a subrange, so smaller backend verification units do not reduce the bytes this client fetches.

7.2 Publication and recovery

Publication validates durable objects before installing a lower binding and acquiring reader ownership. Recovery checks generations and leases; a manifest identity alone conveys no write authority. The connected recovery path reuses the same storage allocation and checks worker identity and scoped metadata authority. Catalog operations provide quiescence, verification, retention, and atomic version transitions. Cross-worker handoff requires runtime orchestration to connect them to session checkpoints and tool reopening.

7.3 Layout support

The ordinary uploader defaults to $g = 4\,\text{MiB}$ within 64 MiB chunks. Readers use the same geometry; incompatible configurations are rejected. The client size remains a fixed configuration.

Encoding hints and new verification framing require compatible client and backend support, with separate admission gates disabled by default in the inspected backend. Effective layouts are persisted; configuration changes do not re-encode existing objects. The ordinary uploader passes hints. Direct workspace writes and the snapshot producer use defaults. Direct writes hold fragments capped at the automatic policy's no-hint threshold.

The current policy omits hints for $L_o \leq 1\,\text{MiB}$ and requests $B = 1\,\text{MiB}$ otherwise. The inspected backend already defaults to $B = 1\,\text{MiB}$, so the policy can leave geometry unchanged while changing client fetch and cache behavior. Evaluation distinguishes these effects using actual (g, L_o, B, q) and the path's fetch policy.

8 Evaluation Plan

The evaluation tests each mechanism in a workload that exposes its expected benefit, then asks whether those benefits improve fork, migration, and concurrent execution. The formal study remains to be conducted. Section 2.3 reports the completed request characterization; an exploratory replay is separately archived in the companion experiment repository.

8.1 Systems, inputs, and accounting

JuiceFS is the external distributed-file-system baseline [8]. Internal comparisons use ordinary volumes, native-metadata workspaces, and packed-metadata workspaces. Mechanism ablations keep the execution engine and unrelated policies fixed; comparisons between complete paths are reported separately. Resource pools match CPU, memory, storage, and metadata-service budgets. Concurrent comparisons require independent services; runs sharing a service execute sequentially. Software revisions, effective layouts, placement, and supported operations are recorded.

Inputs include coding and build/test traces, a million-file collection with 4 KiB payloads, and fixed collections shared by concurrent readers. The companion protocols specify generation, schedules, and repetitions. ML dataset access, repeated publication, retention, and scale studies extend the configured replay and small-file experiments.

Accounting covers preparation, execution, validation, retention, and reclamation, including failed attempts. Shared preparation is counted once per baseline; object residence continues until reclamation. CPU time, transferred bytes, and monetary cost retain separate units. Batch duration is measured directly, with queueing included, rather than summed from overlapping tasks. Replay and actual agent execution are separate studies.

8.2 Overlay fork and retained branches

RQ1. How much state and work does overlay fork save without packing metadata? Many children start from one revision and modify a small part of its tree. A native-metadata workspace is compared with an eagerly materialized private view in the same engine, preserving the API, payload sharing, caches, and durability semantics. An ordinary copied volume provides a separate system-level comparison.

Increasing fan-out tests preparation sharing. Star, branching, and chained histories use equal fork budgets to expose the cost of descendants of modified parents. Measurements include baseline capture, creation to first correct operation, first durable edit, save latency, private records, and uploaded bytes. Readback checks inheritance and parent/sibling isolation. Retaining, reopening, and deleting states measures accumulated metadata and actual reclamation; admission failures and outstanding cleanup remain visible.

8.3 Read-only metadata service

RQ2. When does packing a fixed small-file collection reduce database load enough to repay construction and publication? Concurrent readers repeatedly access a fixed collection while other workspaces update the same metadata service. The pure metadata comparison changes native versus packed representation within one engine, holding payload layout and caches identical. A separate comparison permits each complete format to use its own data layout. Standalone read-only mounts isolate the representation from upper checks.

Native version capture and binding are timed alongside the additional construction, upload, validation, and publication required for an unprepared packed version. First and later forks separate preparation from reuse. Equal-use comparisons vary consumers, version churn, and post-use retention to identify resource-specific break-even regions. Query offload and persistent database reduction are measured separately: retained source records still occupy space.

Positive and negative lookups, inode attributes, neighboring names, enumeration followed by attributes, and lookup followed by reads exercise different locality. Cold, index-warm, and metadata-warm runs declare supported cache budgets. Counters separate lower-database, private-upper, and control traffic from index, joint-page, descriptor, and payload requests. Latency, database CPU and memory, persistent bytes, transfers, decoding, and verification reveal where work moves. A fixed-rate mutable-workspace probe measures interference through its queue, deadline completion, and tail latency.

An ML-oriented case separates dataset discovery from sample loading: fresh enumeration versus sample-list reuse,

then ordered versus shuffled sample access, with outputs outside the fixed dataset. Larger-file inputs test the role of per-file metadata. Compatible Nydus or SOCI readers provide read-only layout comparisons at matched content, integrity, cache, and payload settings. An immutable sorted-KV control isolates joint-page organization. These comparisons charge construction, updates, and residence as well as query service.

8.4 Object formation and backend work

RQ3. How do client and backend granularity affect physical work, write completion, and repair? Small reads within larger objects expose excess fetching and verification; scans test the request overhead of smaller units. Identical writes compare client sizes of 256 KiB, 1 MiB, and 4 MiB at fixed backend geometry. Fixed finalized objects and S3 requests then compare admitted encoding-block hints. Verification policies are varied separately at a fixed effective encoding block size. Range and whole-object fetching have matched cache, coalescing, and prefetch controls.

Boundary-crossing reads, shared and disjoint keys, overwrites, and supported degraded reads test the resulting tradeoffs. Object counts, finalized lengths, HTTP and device bytes, checksum and encoding CPU, durable-write latency, and repair traffic explain performance. Actual (g, L_o, B, q) is recorded. An unchanged (B, q) provides no backend-layout intervention, even if the policy changes client fetching. HTTP bytes do not substitute for missing physical-read counters.

8.5 Migration and recovery

RQ4. How much work does migration require before and after publication? An agent resumes on another worker while neighboring tasks continue. Only paths that reopen the existing workspace enter this comparison; fresh-only paths are evaluated for creation instead. Paired stay/move runs vary the active population and migrated fraction. Source and target host identities establish that a move crosses workers; JuiceFS uses its native directory reopen operation.

Handoff occurs at a completed-tool boundary, with either a published version or pending modifications. Interruption includes queueing, draining, capture, any required packing, publication, and authority transfer. Cold and warm targets measure first correct operation, continuation cost, transfers, and neighboring tasks' completion and tail latency.

File-state tests check names and bytes. Actual agent tests preserve session history and remaining budget through export/import, require a subsequent valid tool result, and grade final task completion. Published file state does not checkpoint process memory or open handles. Failures before and after publication, disconnected source coordination, and target failure test recovery and stale-writer rejection. Independent readback distinguishes storage-enforced rejection from process termination or application checks.

8.6 Concurrent tasks and analysis

RQ5. Do component savings increase useful capacity at fixed resources? A fixed assignment set runs at increasing concurrency under a common deadline, including queueing. A separate scale study increases workers and assigned work against fixed backend capacity. Placement and replay-engine CPU limits are recorded.

Internal ablations change one supported mechanism; composition is compared with its best supported proper subset. External system comparisons remain separate. Paired blocks report raw outcomes and ranges. Stronger inference uses a sample size chosen from a separate variance pilot; requests and sibling tasks are not independent replicates. Deadline completion includes failures in its denominator. Complete-success makespan is reported only when every assignment succeeds, alongside failed outcomes and measurement gaps.

9 Related Work

Private workspaces. OverlayFS combines upper and lower trees using copy-up and deletion markers [13], and BranchFS adds a fork, explore, and commit lifecycle whose evaluated prototype copies an inherited file when it is first modified [22]. BrewFS keeps the shared baseline immutable and stores an attempt's own names, attributes, and data mappings, so unchanged ranges keep their original references instead of being copied as a whole file. The lower-query interface accepts either native or packed metadata, which is what allows the two mechanisms to be used together or apart. YoloFS emphasizes staging, reversible effects, and permissions [29]; AgentFS combines files, key-value state, and tool-call records in SQLite [4]. These execution-control and observability functions complement the storage mechanisms studied here.

Mutable metadata services. TableFS and IndexFS use table-based layouts for metadata and small files; IndexFS also partitions directories and supports caching and bulk insertion [18, 19]. LocoFS and InfiniFS separate metadata responsibilities to improve distribution and access [12, 14]. Mantle targets hierarchical object metadata, while SwitchFS coordinates asynchronous directory updates [11, 26]. Their partitioning and caching reduce work in mutable metadata services. Packed Metadata changes the representation and service path of a selected fixed subtree: supported queries use bounded object pages, while private-upper checks, updates, and lifetime control retain their existing services. Its contribution is evaluated through the database work displaced and the object-access and publication costs introduced.

Private namespaces and immutable logs. DeltaFS combines private namespaces with object-backed metadata. Jobs publish mutation logs as SSTables; subsequent jobs select and, when needed, merge snapshots. Job-local servers and compaction reduce dependence on a globally synchronized metadata service [28]. BrewFS maintains a current-value private upper over a pinned lower. Packing is a separate step that organizes fixed records by directory and name range, co-locating attributes and data locators. A native-metadata workspace avoids this construction while still paying for version capture, private updates, and ownership. The distinction concerns the update representation, query organization, and choice of when to construct an immutable read representation.

Read-only distribution and indexing. CernVM-FS publishes content-addressed trees through nested SQLite catalogs and downloads metadata and data on demand [2]. It is a close precedent for serving large immutable collections outside a mutable metadata service. OCI defines layered filesystem archives, including compressed tar representations [16]. Lazy loading is supported by systems such as SOCI [20]. Nydus separates filesystem metadata from chunked data and provides caching and integrity verification [15]. Slacker and DADI also reduce image distribution and startup work through demand-driven access [5, 10].

CoFS exploits a fixed image namespace by building minimal perfect hashes and storing metadata in a dense array, so that lookup happens in the kernel and cached-data paths avoid userspace interposition [23]. Its setting is the container image, while BrewFS addresses the metadata service that agents use throughout execution and keeps mutable workspace operations in their existing services. Its packed representation groups directory entries, attributes, and data locators into bounded pages reached through authenticated range indexes, and those groups are the remote metadata access units independently of the physical containers that hold them. The relevant comparison with these immutable distributions is the work required to locate, fetch, retain, and reuse those records for a version that is shared by many readers.

Object granularity. Ceph and Tectonic separate metadata and storage services [17,25]. Azure's erasure coding reduces repair traffic, Morph studies redundancy transitions over a file's lifetime, and Okapi separates striping from redundancy grouping [1, 6, 9]. BrewFS exposes object formation and backend layout as separate filesystem choices. Client block size g controls formation, backend block size B controls encoding, and q controls shard verification and read coverage. Finalized object length informs a hint, the backend validates and persists the effective layout, and persisted geometry governs reads and repair. The design retains the backend's erasure code and exposes these distinct units to the filesystem, so that a volume with small reads can form and verify objects that suit them.

10 Conclusion

BrewFS gives agents private working state while allowing shared inputs to use different metadata representations and storage layouts. Range mappings retain unchanged data, packed read-only versions move supported queries away from the mutable database, and separate object and backend units control the work induced by partial reads. These mechanisms can be selected independently for the state and access pattern of a workload. The eight-run characterization supplies request mixes and timing for evaluation. The planned experiments will quantify preparation and reuse costs, worker handoff, and useful capacity under concurrent execution.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC application software and conditions databases using the CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[4] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[5] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[6] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[8] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[9] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[10] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[11] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[12] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[13] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[14] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[15] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[16] Open Container Initiative. Image layer filesystem changeset. OCI Image Format Specification. Accessed 2026-10-26.

[17] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[18] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[19] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[20] SOCI Snapshotter Contributors. SOCI snapshotter. Official project documentation. Accessed 2026-10-26.

[21] Torchvision Contributors. DatasetFolder and ImageFolder. Torchvision documentation and implementation. Accessed 2026-10-26.

[22] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[23] Li Wang, Jinxu Du, Yang Yang, Qingbo Wu, Tao Liu, and Haoze Wu. CoFS: A filesystem for fast container startup. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 415–423. USENIX Association, 2026.

[24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao,

Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[25] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[26] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[28] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[29] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.